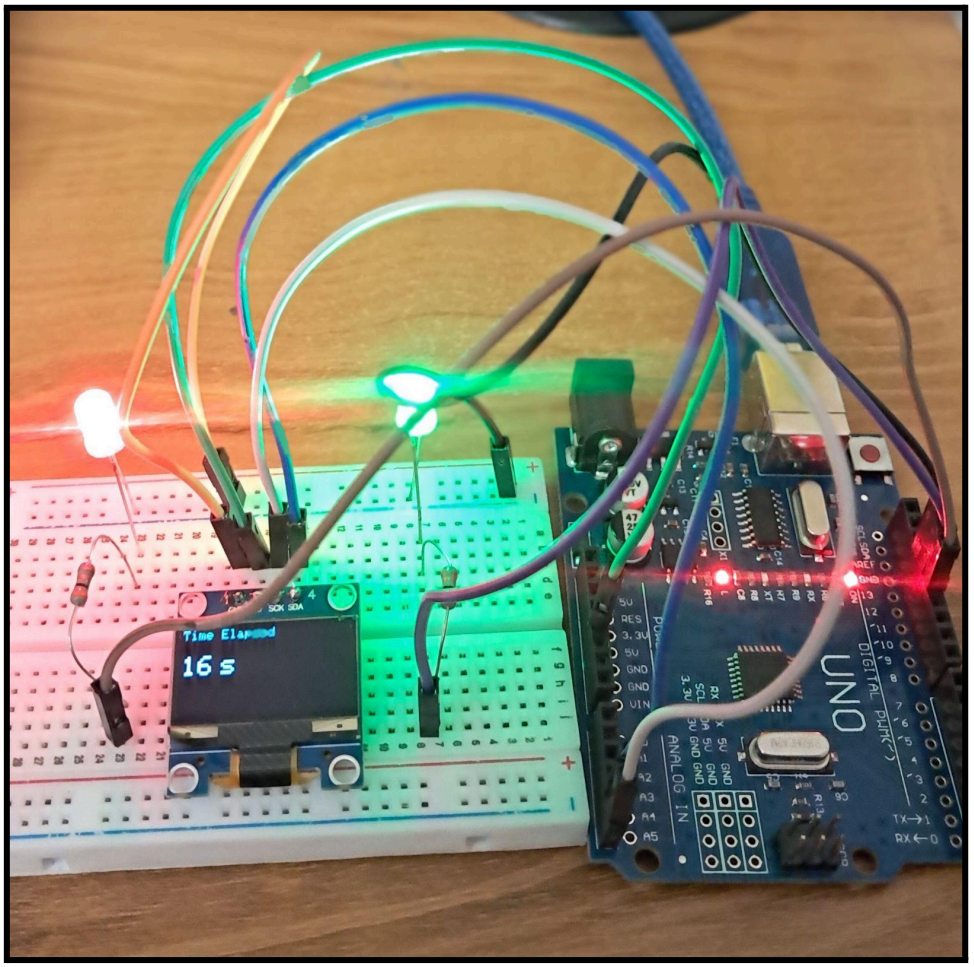
PROJECT 12 REPORT

Non-Blocking Programming using millis( )

Name : Priyaansh Tiwari
Department : Electrical and Electronics Engineering
Date : August 2026
Duration :2 days



1. Objective

The objective of this project was to understand non-blocking programming using the **millis( )** function in Arduino.

The project focused on understanding the difference between traditional delay( ) based programming and millis( ) based timing and learning how multiple independent tasks can run simultaneously without blocking the main program.

2. Components Used

Component	Quantity
Arduino Uno R3	1
0.96 inch OLED Display	1
LEDs(Red,Green,Yellow)	3
Resistors(300Ω)	3
Breadboard	1
Jumper Wires	Multiple
USB Cable	1

3. Software Used

- Arduino IDE
- Adafruit GFX Library
- Adafruit SSD1306 Library
- Wire Library
- Windows Laptop

4. Theory

In traditional Arduino programming, the delay( ) function pauses the execution of the program for a specified amount of time.

During this period, the Arduino cannot proceed normally with other tasks in the program.

The millis( ) function provides an alternative method of timing. It allows the Arduino to continuously check how much time has passed instead of blocking the program.

The basic timing logic used in the project was:
if(currentTime - previousTime >= interval)

- **currentTime :** stores the current value returned by millis( ).
- **previousTime :** stores the time when the task was last performed.
- **currentTime - previousTime :** calculates the elapsed time.

The task executes when the elapsed time reaches the required interval.

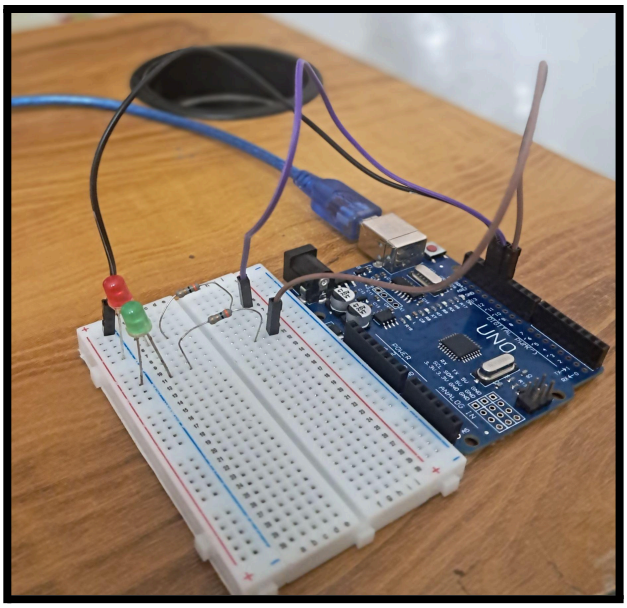
5. Tasks Performed

➢ Single LED using millis( )

A single LED was programmed to blink using millis( ) instead of delay( ).

The Arduino continuously checked the elapsed time and toggled the LED whenever one second had passed.

➢ Two LEDs with Independent Timers

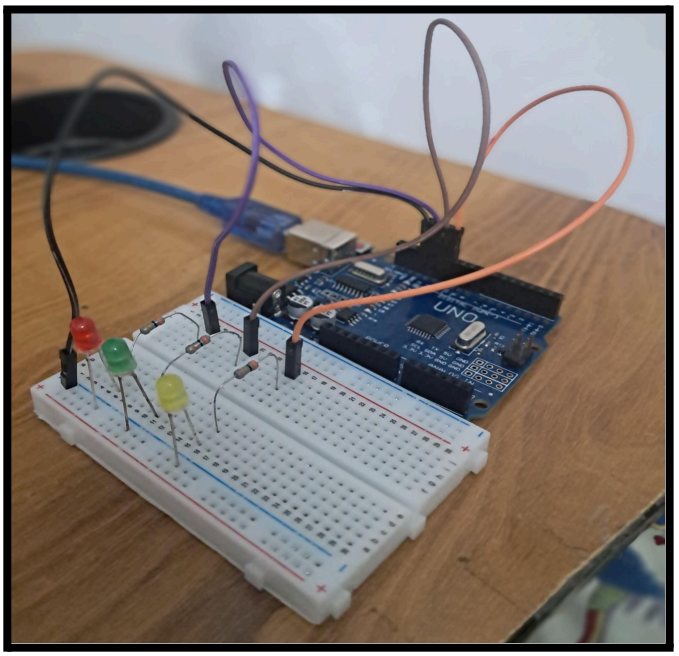


Video

Two LEDs were connected and programmed using separate timing variables.

This demonstrated that multiple timing operations can be handled independently within the same loop( ).

➢ Three LEDs with Independent Timers



Video

The concept was extended to three LEDs.

Each LED had its own previousTime variable. All three LEDs operated simultaneously without using delay( ).

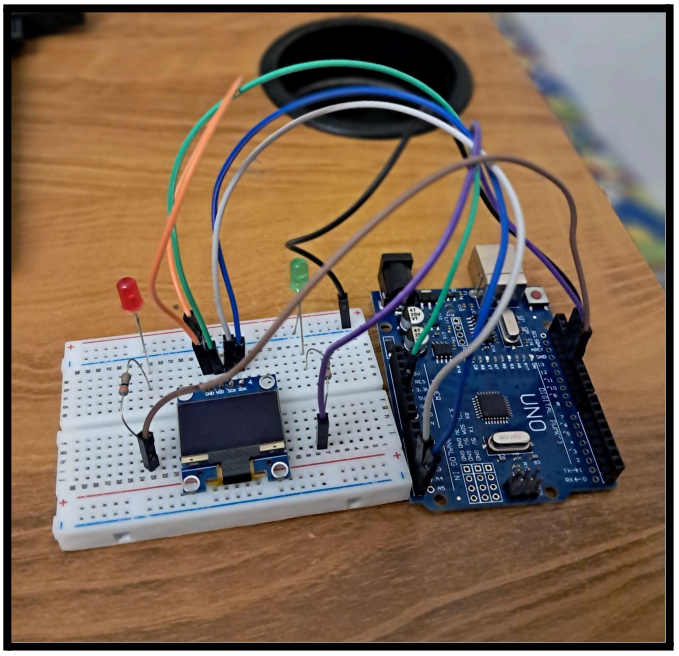
➢ Comparison between delay( ) and millis( )

The same LED setup was implemented using delay( ) to observe the difference between the two approaches.

With delay( ), the LEDs followed a sequential pattern because the Arduino waited during every delay period. With millis( ), all three LEDs operated independently because the Arduino continuously checked the timing conditions rather than stopping the execution.

This task demonstrated the practical difference between **blocking** and **non-blocking** timing.

➢ LEDs and OLED Running Simultaneously



Video

The final task combined the millis( ) timing concept with the OLED display.Two LEDs and an OLED display were operated simultaneously.

The OLED displayed a continuously increasing elapsed time counter while the two LEDs continued blinking independently.

6. Learning Outcomes

During this project, the following concepts were learned:

- millis( ) function
- Non-blocking programming
- Difference between delay( ) and millis( )
- Elapsed time calculation
- unsigned long data type

- Independent timing using multiple previousTime variables
- Running multiple tasks inside the same loop( )
- LED toggling without delay( )
- Updating an OLED display using timed execution
- Combining OLED display control with non blocking timing

7. Problems Faced

I. Initially, the millis( ) LED blink appeared identical to the LED blink created using delay( ). **(Multiple LEDs with different timing intervals were introduced to demonstrate that millis( ) allows several tasks to operate independently)**

8. Observations

- A single LED using millis( ) visually behaved similarly to a normal delay( ) blink.
- The difference became clear when multiple tasks were executed simultaneously.
- With delay( ), the Arduino followed a sequential execution pattern.
- With millis( ), multiple tasks could operate independently.
- Each task required its own previousTime variable.
- The OLED could be updated at a fixed interval while LEDs continued operating independently.
- The loop( ) continuously checked the timing conditions instead of being blocked by delay( ).

9. Applications and Future Scope

The non-blocking timing concepts learned in this project can be used in Embedded control systems,LED and indicator systems,Motor control and Systems requiring multiple simultaneous timed operations.

The concepts learned in this project will be used in future projects to operate multiple sensors, displays, actuators and communication systems simultaneously without relying heavily on delay( ).

10. Conclusion

This project successfully introduced non-blocking programming using the Arduino millis( ) function.

A direct comparison with delay( ) demonstrated that delay( ) causes sequential and blocking execution, while millis( ) allows the Arduino to continuously execute the loop( ) and independently manage multiple timed tasks.

The project established the foundation for developing more responsive and complex embedded systems where multiple operations need to run simultaneously without blocking one another.